



Caching and Performance Decisions

May 2026

Recap



Day 1-2

JDBC, transactions, mini-ORM



Day 3-4

Hibernate, relationships, fetch strategies



We traded boilerplate for ORM convenience



Today

Caching and performance decisions

Time planning



Starting time: 9.30



Breaks: ...



End time: 16.30



Section 1

Why ORMs need caching

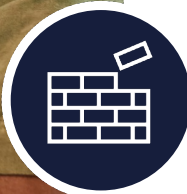
Why repeated reads hurt



Every query has real cost



Network hop plus database compute



ORM must still materialize objects



Repeating reads repeats all that work



What caching solves



Keep recent data close to app



Reuse known entities instead of querying



Faster responses, lower DB load



Preserve object identity in memory

Why ORMs need cache specifically



ORMs manage object graphs, not rows



Same row reached by multiple paths



Cache prevents duplicate object instances



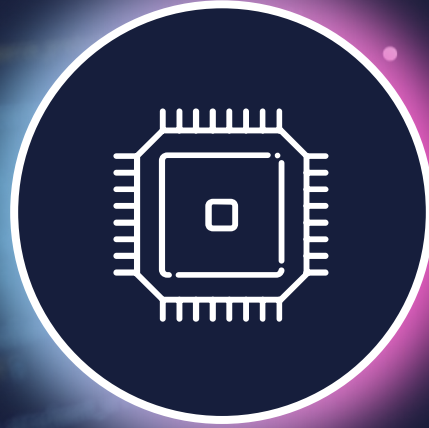
Cache prevents duplicate database queries

The trade-off

Speed vs correctness risk

- Faster reads
- Less database pressure
- Higher memory usage
- Invalidation logic becomes necessary
- Stale data risk must be managed





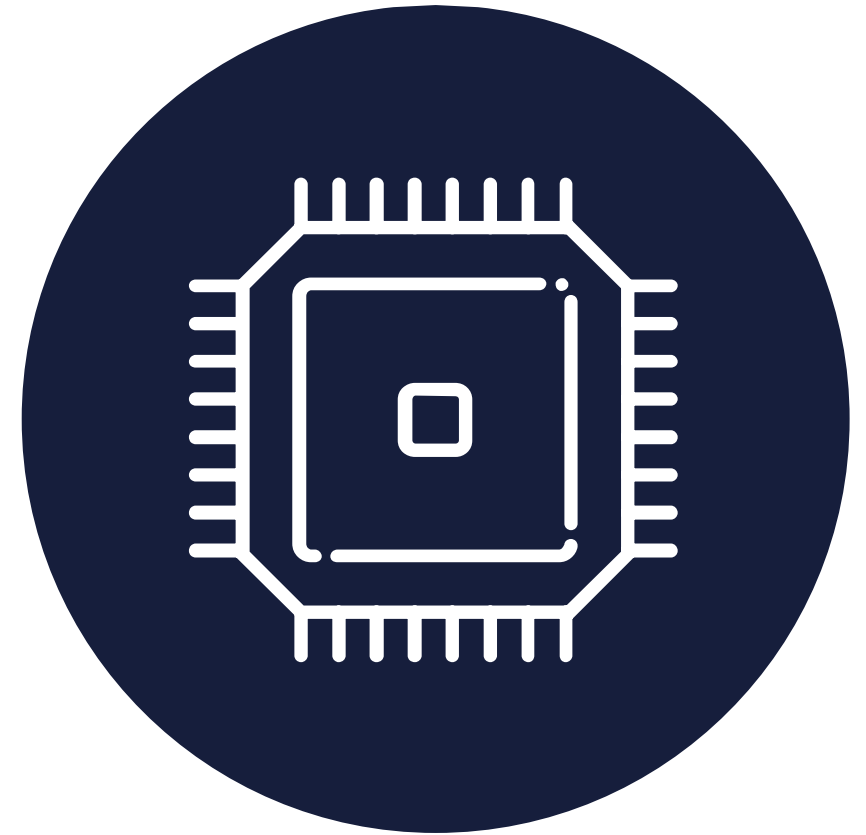
Section 2

First-level Gate



What is first-level cache?

- Persistence-context cache (EntityManager/Session)
- Lives inside the persistence context
- Caches fetched and persisted entities as managed state
- Disappears when context closes



Lookup flow in L1 cache



Fetch by id checks persistence context first



Cache hit returns entity, no SQL



Cache miss triggers database query



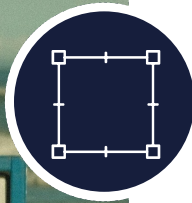
Loaded entity stored as managed entity

Identity guarantee inside a persistence context

- Same id returns same Java instance
- `find(...) == find(...)` is guaranteed true
- Changes remain visible in same context
- Across contexts identity is not shared



L1 cache limits



Works only within one persistence context



Typical request has its own persistence context



No sharing across separate requests



Does not solve multi-request hot data

Persistent context controls

- Usually managed fully by Hibernate
- `entityManager.clear()` detaches all managed entities
- `entityManager.detach(entity) / session.evict(entity)` detaches one
- Detached entities lose dirty checking and lazy loading
- Useful for batch and consistency edge cases

Mini exercise

Persistence context cache in action

- Fetch course by id, inspect SQL
- Fetch same id again, expect no SQL
- Compare with equality operator
- Query same course by other path
- Open new persistence context, SQL appears again





Section 3

Second-level cache and strategies

Why L1 is not enough

- L1 cache is per-session only
- Repeated requests still hit database
- Hot entities can create read storms
- That's why we need cache beyond session boundaries



What is second-level cache?



Shared cache across sessions



Managed by Hibernate `SessionFactory`



Optional and explicitly configured



Backed by cache providers



L1 → L2 → Database read path

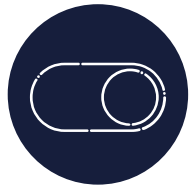
- Check L1 cache first
- Then check L2 cache
- Then query database on miss
- Populate caches after database fetch



Enabling L2 cache in Hibernate



Add cache provider dependency



Enable second-level cache property



Configure JCache region factory (...`region.factory_class=jcache`)



Configure provider/regions and mark entities cacheable

Cache concurrency strategies



Read-only for immutable reference data



Read-write for mutable common entities (common choice)



Nonstrict read-write allows brief staleness



Transactional requires JTA and transactional provider

Invalidation and coherency

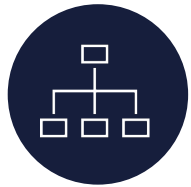
- Hibernate invalidates on ORM-managed entity updates
- Bulk HQL/JPQL and external SQL can cause staleness
- TTL (time to live) limits stale lifetime
- Manual eviction and events can help
- Cache correctness depends on all writes flowing through known paths and explicit invalidation mechanisms



Caching relationships and collections



Collections can be cached separately (but use sparsely)



Child entities still need own caching



Eviction complexity grows with relationships



Prefer caching stable individual entities

Discussion

What to cache in school domain?

- Good candidates: stable reference-like data
- Risky candidates: rapidly changing data
- Debate correctness versus speed tolerance
- Decide by access frequency and volatility

Mini exercise

Conceptual L2 Cache Behavior

- L2 disabled: many requests, many queries
- L2 enabled: first miss then many hits
- Update invalidates cached entity
- Native SQL update can bypass invalidation





Section 4

ORM performance pitfalls and optimization around



Pitfall overview



Caching helps but can hide issues



Correctness and memory can degrade



Query shape still drives performance



Treat cache as an optimization tool

Pitfall 1 and 2:

Stale data and over-caching



External writes can stale cached values



Users may see outdated information



Unlimited cache growth pressures memory



Apply TTLs and size limits early

Pitfall 3 and 4:

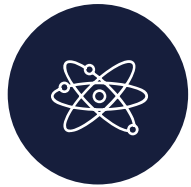
Row multiplication and overfetching



Multiple join-fetches can multiply result rows



ORM spends time deduplicating large sets



Full graphs loaded for tiny UI needs



Use projections and narrower fetch plans

Pitfall 5 and 6:

Indexes and distributed coherency



Cache misses depend on database speed



Missing indexes make misses very expensive



Multi-server local caches can diverge



Distributed cache improves coherence

Pitfall 7 and 8:

Cold start and framework differences



Cold start causes initial cache misses



Warmup can improve steady-state latency



Warmup increases startup work



Caching behavior differs across ORMs

Discussion

When should you cache?

- Cache stable, frequently read data
- Avoid caching rapidly changing critical data
- Consider memory and stale-data tolerance
- Decide using measurements, not intuition



Caching is not the whole story

- Query design still matters most
- Access patterns drive total cost
- Caching amplifies good query shapes
- Caching cannot fix poor modelling
- So good fundamentals first, cache second



Query optimization tactics



Use projections for read-only screens



Limit join-fetches to needed paths



Shape queries to concrete use cases

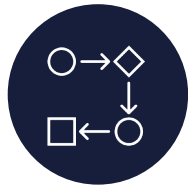


Revisit hot paths before adding cache

Database-side optimization matters



Index foreign keys and lookup columns explicitly



Validate query plans for hot queries



Keep DB statistics current



ORM speed depends on DB health

Monitoring and profiling workflow



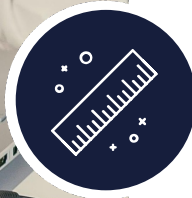
Inspect Hibernate SQL logs in development



Use app profiler for hot code paths



Check database slow query logs



Measure first, optimize second

The 80/20 rule in practice

- Few issues cause most latency
- Focussing on hotspots prevents over optimization
- One N+1 fix can beat broad caching
- Prioritize highest-impact bottlenecks first
- Avoid premature micro-optimizations



Mini exercise

Identify the real bottleneck

- Analyze the problem
- Try different solutions
- Compare and contrast different solutions



Key takeaways



- L1 cache gives per-context identity and reuse
- L2 cache reduces cross-session database reads
- Invalidation strategy determines correctness safety

This afternoon

Work on your project + presentation...